Herencia, constructores y sobreescritura de métodos

Proyecto: CielosLimpios
Autor: isacmn45
Fecha: 5 de noviembre de 2026

Objetivo: implementar la jerarquía Persona → Estudiante → Docente mediante constructores encadenados y métodos sobreescritos, dentro del mismo proyecto del reto Cielos Limpios.



Estudiante hereda de Persona; Docente hereda de Estudiante. Cada constructor de una subclase llama a super(...) para inicializar los datos de su clase padre.

Las tres clases definen presentar() y realizarActividad(). Las subclases usan @Override; presentar() amplía la respuesta heredada con super.presentar(). Main utiliza Persona[] para demostrar que se ejecuta el método del tipo real de cada objeto.

Se conservan las tres clases de drones y su simulación. La ejecución normal muestra ambos ejercicios. La opción --personas muestra únicamente la nueva jerarquía.

Código y acceso en la nube

Repositorio: github.com/isacmn45/CielosLimpios

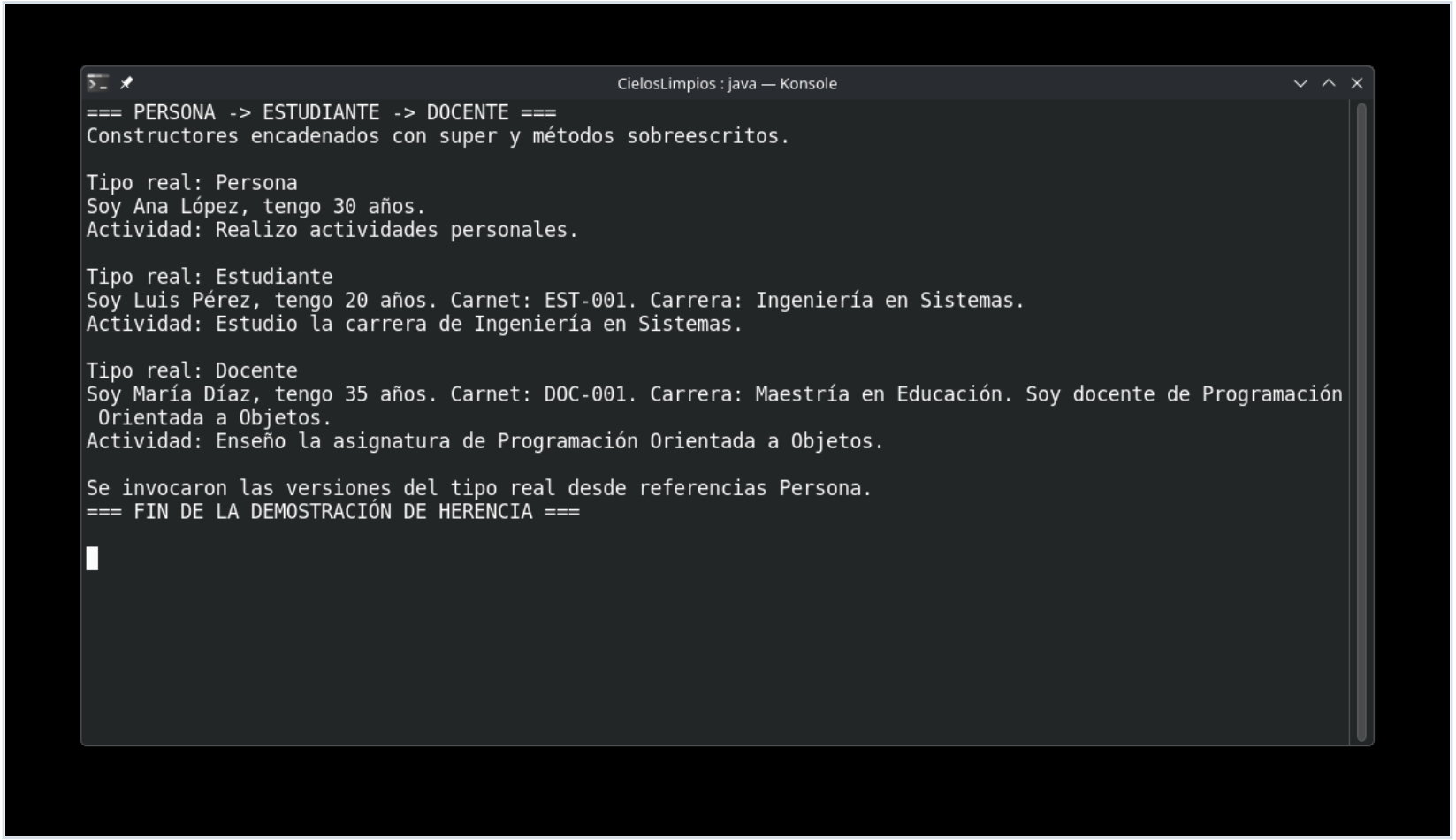
Abrir directamente las clases del ejercicio

Abrir las pruebas del proyecto

El informe contiene las siete clases de aplicación completas y las capturas de los resultados. La batería de comprobaciones incluye 18 grupos de pruebas.

Resultado de la jerarquía

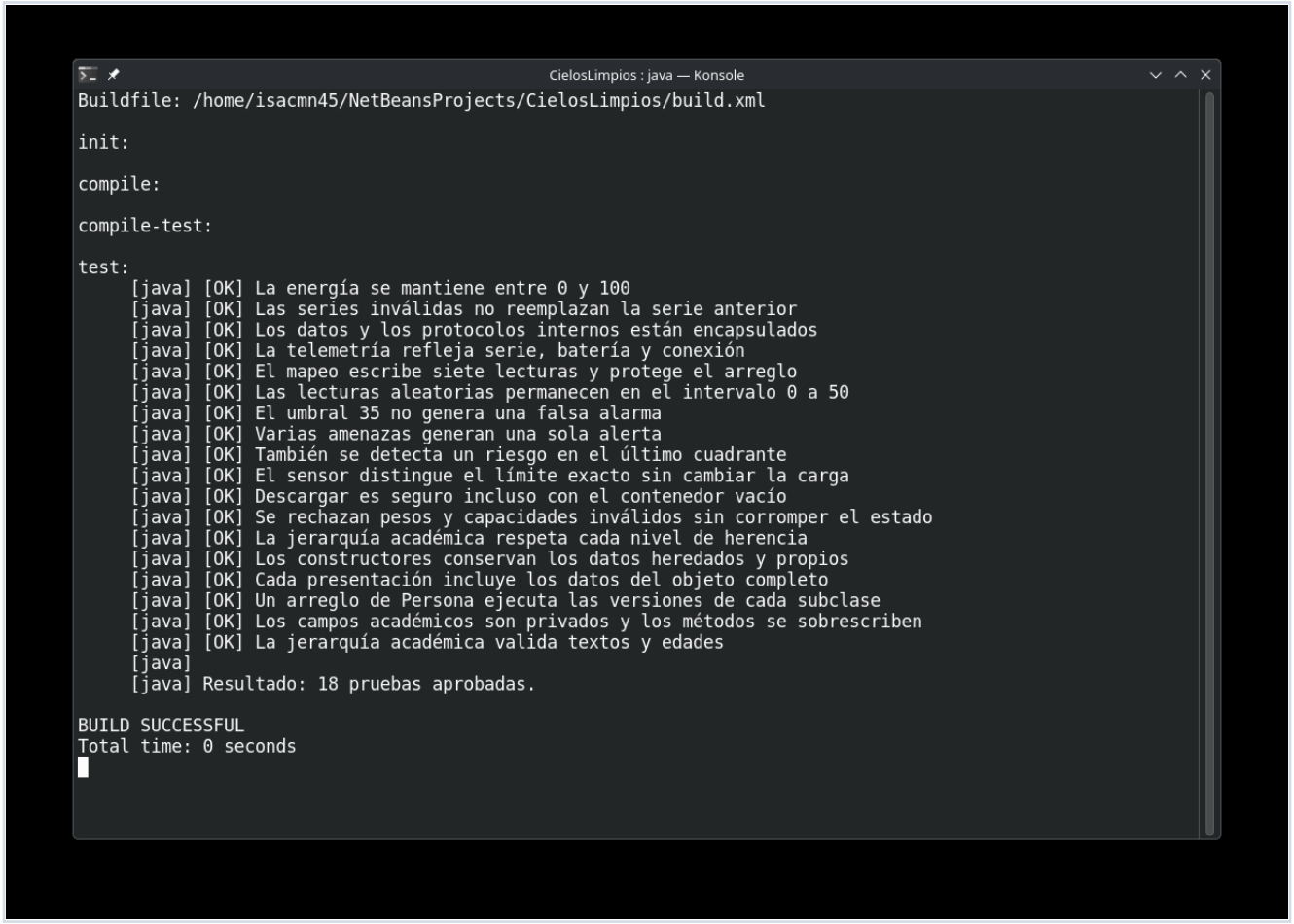
Figura 1. Captura de consola de la ejecución del ejercicio de personas.



La presentación del docente incluye los datos heredados de Persona y Estudiante. Las actividades son distintas para los tres objetos, aunque el recorrido utiliza referencias de tipo Persona.

Comprobaciones del proyecto

Figura 2. Captura de consola de las comprobaciones de ambos ejercicios.



Resultado: 18 grupos de pruebas aprobados. Se comprueban la jerarquía exacta, los datos de los constructores, la sobreescritura, el despacho dinámico, la encapsulación y las validaciones, además de las reglas del reto de drones.

Persona.java

Clase base: nombre, edad y comportamiento general.

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package com.mycompany.cieloslimpios;

/**
 * Clase base que reúne los datos comunes de una persona.
 *
 * @author isacmn45
 */
public class Persona {

    private final String nombre;
    private final int edad;

    public Persona(String nombre, int edad) {
        this.nombre = validarTexto(nombre, "El nombre");
        if (edad < 0) {
            throw new IllegalArgumentException("La edad no puede ser negativa.");
        }
        this.edad = edad;
    }

    public String getNombre() {
        return nombre;
    }

    public int getEdad() {
        return edad;
    }

    public String presentar() {
        return "Soy " + nombre + ", tengo " + edad + " años.";
    }

    public String realizarActividad() {
        return "Realizo actividades personales.";
    }

    protected static String validarTexto(String texto, String campo) {
        if (texto == null || texto.trim().isEmpty()) {
            throw new IllegalArgumentException(campo + " no puede estar vacío.");
        }
        return texto.trim();
    }
}
```

Estudiante.java

Hereda de Persona y añade los datos académicos.

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this li
cense
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package com.mycompany.cieloslimpios;

/**
 * Especialización de Persona con los datos académicos de un estudiante.
 *
 * @author isacmn45
 */
public class Estudiante extends Persona {

    private final String carnet;
    private final String carrera;

    public Estudiante(String nombre, int edad, String carnet, String carrera) {
        super(nombre, edad);
        this.carnet = validarTexto(carnet, "El carnet");
        this.carrera = validarTexto(carrera, "La carrera");
    }

    public String getCarnet() {
        return carnet;
    }

    public String getCarrera() {
        return carrera;
    }

    @Override
    public String presentar() {
        return super.presentar() + " Carnet: " + carnet + ". Carrera: " + carrera + ".";
    }

    @Override
    public String realizarActividad() {
        return "Estudio la carrera de " + carrera + ".";
    }
}
```

Docente.java

Hereda de Estudiante y añade la asignatura que imparte.

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this li
cense
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package com.mycompany.cieloslimpios;

/**
 * Tercer nivel de la jerarquía solicitada: Persona, Estudiante y Docente.
 *
 * @author isacmn45
 */
public class Docente extends Estudiante {

    private final String asignatura;

    public Docente(String nombre, int edad, String carnet, String carrera,
            String asignatura) {
        super(nombre, edad, carnet, carrera);
        this.asignatura = validarTexto(asignatura, "La asignatura");
    }

    public String getAsignatura() {
        return asignatura;
    }

    @Override
    public String presentar() {
        return super.presentar() + " Soy docente de " + asignatura + ".";
    }

    @Override
    public String realizarActividad() {
        return "Enseño la asignatura de " + asignatura + ".";
    }
}
```

Main.java

Demuestra el despacho dinámico y conserva la simulación de drones.

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package com.mycompany.cieloslimpios;

import java.util.Locale;
import java.util.Random;

/**
 * Consola central de la operación: las dos flotas se gestionan por separado.
 *
 * @author isacmn45
 */
public class Main {

    public static void main(String[] args) {
        demostrarJerarquiaPersonas();
        if (args.length > 0 && "--personas".equals(args[0])) {
            return;
        }
        Random random = new Random();
        System.out.println("=== OPERACIÓN CIELOS LIMPIOS | AÑO 2030 ===");

        DronReconocimientoTactico[] escuadronTactico = {
            new DronReconocimientoTactico("TAC-001", 100, true),
            new DronReconocimientoTactico("TAC-002", 80, true),
            new DronReconocimientoTactico("TAC-003", 65, true)
        };

        DronCargaAmbiental[] escuadronPesado = {
            new DronCargaAmbiental("CAR-001", 90, true, 20.0),
            new DronCargaAmbiental("CAR-002", 60, true, 35.0)
        };

        System.out.println("\n--- MISIÓN DEL ESCUADRÓN TÁCTICO ---");
        for (DronReconocimientoTactico dron : escuadronTactico) {
            dron.generarMapeoEstocastico();
            dron.analizarAmenazas();
        }

        System.out.println("\n--- MISIÓN DEL ESCUADRÓN PESADO ---");
        for (DronCargaAmbiental dron : escuadronPesado) {
            double pesoNuevo = 1 + random.nextInt(50); // De 1 a 50 kg.
            System.out.printf(Locale.ROOT, "[%s] Desecho encontrado: %.2f kg.%n",
                    dron.getCodigoSerie(), pesoNuevo);
            dron.engancharDesecho(pesoNuevo);
            dron.descargarContenedorBase();
        }

        System.out.println("\n--- PRUEBA DEL SEGURO DE ENERGÍA SUPERIOR ---");
        escuadronTactico[0].setNivelEnergia(125); // Se limita a 100%.

        System.out.println("\n--- DESGASTE POST-MISIÓN Y TELEMETRÍA FINAL ---");
        // El consumo de la segunda unidad supera su batería para probar el límite 0.
        int[] desgasteTactico = {15, 120, 10};
        for (int i = 0; i < escuadronTactico.length; i++) {
            DronReconocimientoTactico dron = escuadronTactico[i];
            dron.setNivelEnergia(dron.getNivelEnergia() - desgasteTactico[i]);
            dron.setVinculadoRed(dron.getNivelEnergia() > 0);
            dron.transmitirEstadoRed();
        }

```

Main.java (continuación 2)

Demuestra el despacho dinámico y conserva la simulación de drones.

```
        int[] desgastePesado = {20, 100};
        for (int i = 0; i < escuadronPesado.length; i++) {
            DronCargaAmbiental dron = escuadronPesado[i];
            dron.setNivelEnergia(dron.getNivelEnergia() - desgastePesado[i]);
            dron.setVinculadoRed(dron.getNivelEnergia() > 0);
            dron.transmitirEstadoRed();
        }

        System.out.println("\n=== SUPERVISIÓN FINALIZADA ===");
    }

    private static void demostrarJerarquiaPersonas() {
        System.out.println("=== PERSONA -> ESTUDIANTE -> DOCENTE ===");
        System.out.println("Constructores encadenados con super y métodos sobreescritos.");

        Persona[] personas = {
            new Persona("Ana López", 30),
            new Estudiante("Luis Pérez", 20, "EST-001", "Ingeniería en Sistemas"),
            new Docente("María Díaz", 35, "DOC-001", "Maestría en Educación",
                    "Programación Orientada a Objetos")
        };

        for (Persona persona : personas) {
            System.out.println("\nTipo real: " + persona.getClass().getSimpleName());
            System.out.println(persona.presentar());
            System.out.println("Actividad: " + persona.realizarActividad());
        }
        System.out.println("\nSe invocaron las versiones del tipo real desde referencias Persona
.");
        System.out.println("=== FIN DE LA DEMOSTRACIÓN DE HERENCIA ===\n");
    }
}
```

Dron.java

Clase base del reto original: energía, conexión y telemetría.

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package com.mycompany.cieloslimpios;

/**
 * Chasis común de todas las unidades de la operación Cielos Limpios.
 *
 * @author isacmn45
 */
public class Dron {

    private String codigoSerie;
    private int nivelEnergia;
    private boolean vinculadoRed;

    public Dron(String codigoSerie, int nivelEnergia, boolean vinculadoRed) {
        setCodigoSerie(codigoSerie);
        setNivelEnergia(nivelEnergia);
        this.vinculadoRed = vinculadoRed;
    }

    public String getCodigoSerie() {
        return codigoSerie;
    }

    public void setCodigoSerie(String codigoSerie) {
        if (codigoSerie == null || codigoSerie.trim().isEmpty()) {
            throw new IllegalArgumentException("El código de serie no puede estar vacío.");
        }
        this.codigoSerie = codigoSerie.trim();
    }

    public int getNivelEnergia() {
        return nivelEnergia;
    }

    /** Los valores irregulares se ajustan al porcentaje válido más cercano. */
    public void setNivelEnergia(int nivelEnergia) {
        this.nivelEnergia = Math.max(0, Math.min(100, nivelEnergia));
        if (this.nivelEnergia != nivelEnergia) {
            System.out.printf("[%s] Energía solicitada: %d%%; ajustada a %d%%.%n",
                    codigoSerie, nivelEnergia, this.nivelEnergia);
        }
        calibrarGiroscopioInterno();
    }

    public boolean isVinculadoRed() {
        return vinculadoRed;
    }

    public void setVinculadoRed(boolean vinculadoRed) {
        this.vinculadoRed = vinculadoRed;
    }

    // Private: ni el operador ni las subclases pueden invocar este protocolo.
    private void calibrarGiroscopioInterno() {
        System.out.printf("[%s] Sensores internos reajustados: giroscopio calibrado.%n",
                codigoSerie);
    }

    public void transmitirEstadoRed() {
        System.out.printf("[BALIZAS] Serie: %s | Batería: %d%% | En línea: %s%n",
                codigoSerie, nivelEnergia, vinculadoRed ? "sí" : "no");
    }
}
```

DronReconocimientoTactico.java

Especialización de Dron: siete lecturas y análisis de amenazas.

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package com.mycompany.cieloslimpios;

import java.util.Objects;
import java.util.Random;

/**
 * Unidad especializada en patrullar los siete cuadrantes forestales.
 *
 * @author isacmn45
 */
public class DronReconocimientoTactico extends Dron {

    private final int[] lecturasTermicas = new int[7];
    private final Random random;

    public DronReconocimientoTactico(String codigoSerie, int nivelEnergia,
            boolean vinculadoRed) {
        this(codigoSerie, nivelEnergia, vinculadoRed, new Random());
    }

    // Permite verificar los umbrales con lecturas reproducibles en las pruebas.
    DronReconocimientoTactico(String codigoSerie, int nivelEnergia,
            boolean vinculadoRed, Random random) {
        super(codigoSerie, nivelEnergia, vinculadoRed);
        this.random = Objects.requireNonNull(random, "El generador es obligatorio.");
    }

    public int[] getLecturasTermicas() {
        return lecturasTermicas.clone();
    }

    public void generarMapeoEstocastico() {
        System.out.printf("%n[%s] Escaneando los siete cuadrantes:%n", getCodigoSerie());
        for (int i = 0; i < lecturasTermicas.length; i++) {
            lecturasTermicas[i] = random.nextInt(51); // Incluye 0 y 50.
            System.out.printf("  Sector %d: amenaza térmica %d.%n",
                    i + 1, lecturasTermicas[i]);
        }
    }

    public void analizarAmenazas() {
        for (int i = 0; i < lecturasTermicas.length; i++) {
            if (lecturasTermicas[i] > 35) {
                System.out.printf("[%s] Riesgo detectado en el sector %d: %d.%n",
                        getCodigoSerie(), i + 1, lecturasTermicas[i]);
                emitirAlertaIncendio();
                return; // Se detiene este análisis ante el primer peligro.
            }
        }
        System.out.printf("[%s] Patrulla completada sin amenazas superiores a 35.%n",
                getCodigoSerie());
    }

    private void emitirAlertaIncendio() {
        System.out.printf("[%s] ¡ALERTA ROJA! Posible incendio en el sector%n",
                getCodigoSerie());
    }
}
```

DronCargaAmbiental.java

Especialización de Dron: recolección, sobrecarga y descarga.

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this li
cense
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package com.mycompany.cieloslimpios;

import java.util.Locale;

/**
 * Unidad recolectora de desechos; todos los pesos se expresan en kilogramos.
 *
 * @author isacmn45
 */
public class DronCargaAmbiental extends Dron {

    private final double cargaMaxima;
    private double cargaActual;

    public DronCargaAmbiental(String codigoSerie, int nivelEnergia,
            boolean vinculadoRed, double cargaMaxima) {
        super(codigoSerie, nivelEnergia, vinculadoRed);
        if (!Double.isFinite(cargaMaxima) || cargaMaxima <= 0) {
            throw new IllegalArgumentException("La carga máxima debe ser positiva y finita.");
        }
        this.cargaMaxima = cargaMaxima;
        this.cargaActual = 0;
    }

    public double getCargaMaxima() {
        return cargaMaxima;
    }

    public double getCargaActual() {
        return cargaActual;
    }

    /** Retorna true si el objeto provocaría una sobrecarga; no modifica el dron. */
    public boolean evaluarPesoObjeto(double pesoNuevo) {
        if (!Double.isFinite(pesoNuevo) || pesoNuevo < 0) {
            throw new IllegalArgumentException("El peso debe ser finito y no negativo.");
        }
        return cargaActual + pesoNuevo > cargaMaxima;
    }

    public void engancharDesecho(double pesoNuevo) {
        if (evaluarPesoObjeto(pesoNuevo)) {
            System.out.printf(Locale.ROOT,
                    "[%s] Maniobra abortada: objeto de %.2f kg demasiado pesado. "
                    + "Carga: %.2f / %.2f kg.%n",
                    getCodigoSerie(), pesoNuevo, cargaActual, cargaMaxima);
        } else {
            cargaActual += pesoNuevo;
            System.out.printf(Locale.ROOT,
                    "[%s] Recolección exitosa: %.2f kg. Carga: %.2f / %.2f kg.%n",
                    getCodigoSerie(), pesoNuevo, cargaActual, cargaMaxima);
        }
    }

    public void descargarContenedorBase() {
        System.out.printf(Locale.ROOT, "[%s] Descarga en la base: %.2f kg liberados.%n",
                getCodigoSerie(), cargaActual);
        cargaActual = 0;
    }
}
```